## PH 3120 - Computational Physics Laboratory - I

# 1. Interpolation vs Regression

I. Fundamental difference:

Interpolation estimates unknown values within the range of known data points. Regression models the relationship between variables and used to predict values both within and beyond the data range.

II. Advantages of interpolation:

Interpolation gives very accurate results within the known data range.
It uses all the data points, so the curve passes through each one produsing a smooth and continuous function.

Disadvantages of interpolation:

It can’t predict values outside the known range.
One data point can affect the entire polynomial so that it is easily affected by noise or outliers.

III. Advantages of the regression:

Regression is useful because it can handle noisy data better than interpolation, making it more reliable when data isn't perfect. It also allows to predict values outside the given data range. Regression helps in understanding relationships between variables, showing how one variable changes with another. It's also flexible, with different types like linear or polynomial regression.

Disadvantages of the regression:

If the wrong model is chosen ,regression leads to inaccurate results. Its predictions are not exact for the known data points, unlike interpolation. Also,

it depends on certain assumptions like linearity and normality which may not always be true for real-world data.

IV. Regression is used here because interpolation cannot be used to predict y values beyond the time interval.

V. Effect of outliers and noise on interpolation:

A single outlier can drastically distort the interpolated curve, leading to inaccurate estimates.

VI. Outliers and noise can distort regression results as outliers increase the residual sum of squares resulting skewing the line of best fit.

## 2. Least Squares Regression

I.

(a) Least Squares regression is a method to determine the best-fitted line for the data.Its primary objective is to minimize the total sum of squared errors. (differences between observed and predicted values).

(b) It finds the line of best fit by minimizing the sum of squared differences between the observed data points and predicted values from the line.

(c) The least squares regression line has the equation: $\hat{y} = a + bx$, where $\hat{y}$ is the predicted value, x is the input (independent variable), **a** is the y-intercept, and **b** is the slope. To find the best-fitted line, we calculate the Sum of Squared Errors (SSE), which is the total of all squared differences between actual values ($y_i$) and predicted values ($\hat{y}_i$). To get the values of **a** and **b** that make the SSE as small as possible, we take the partial derivatives of the SSE formula with respect to a and b, set them to zero, and then solve the equations to find the optimal values.

(d) Assumptions of Least Squares Regression

- Relationship between variables is linear.
- Observations are independent.
- Constant variance of residuals.
- Residuals are normally distributed.

(e) RSS Measures the total deviation of the predicted values from actual data.Lower RSS represents the better model fit while a high RSS indicates that the line or curve chosen is not a good representation of the relationship between the variables.

(f) Mean absolute error is the the average absolute difference between actual and predicted values.

(g) Mean absolute error evaluates the quality of a Least Squares Regression model by measuring the average absolute difference between the model's predictions and the actual values.Lower MAE (Mean Absolute Error) implies better prediction accuracy.

II.

a) In linear form:
   T^2 = (4pi^2/k)m

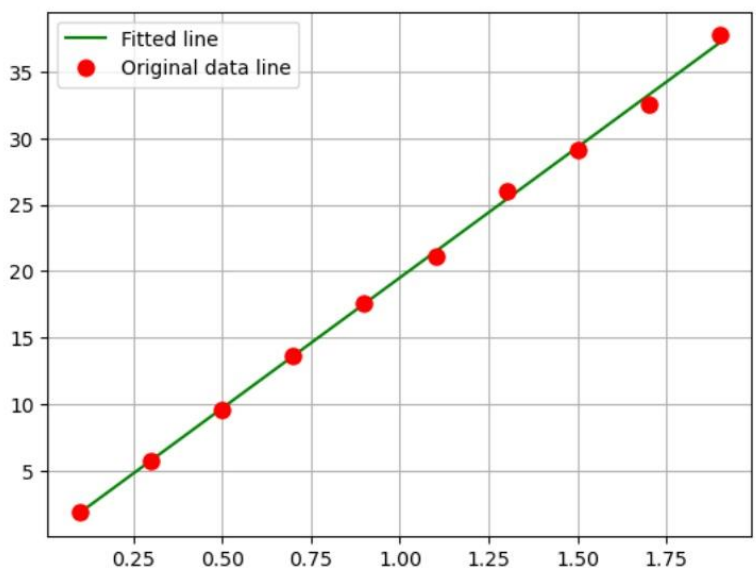
b) Spin constant

## Q2 Least Squares Regression ¶

[14]:
```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0.1,1.9,10)
y = np.array([1.4,2.4,3.1,3.7,4.2,4.6,5.1,5.4,5.7,6.14])

T2= y**2
print("Time square values are:",T2)

coe =np.polyfit(x,T2,1)
fit = np.polyval(coe,x)

plt.plot(x,fit,color ='g',label='Fitted line')
plt.plot(x,T2, 'o',color='r', label ='Original data line', markersize=8)
plt.legend()
plt.grid(True)
plt.show()

k = (((2*(np.pi))**2)/T2)*x
print("Values for sprin constant are:",k)
k1 =np.mean(k)
print("sprin constant (k) : ",k1)
```

```
Time square values are: [ 1.96    5.76    9.61   13.69   17.64   21.16   26.01   29.16   32.49
 37.6996]
```


```
Time square values are: [ 1.96    5.76    9.61   13.69   17.64   21.16   26.01   29.16   32.49
 37.6996]
```


```
Values for sprin constant are: [2.01420498 2.05616758 2.05402797 2.01861887 2.01420498 2.05228069
 1.97316197 2.0307828  2.06566051 1.98964958]
sprin constant (k) :  2.0268759934873635
```

# 3. Polynomial Regression

(I)

(a) Polynomial Regression is a type of regression that models the relationship between variables using a polynomial equation. It differs from Least Squares Regression (linear regression) which uses a straight line to model the data since Polynomial regression uses curved lines and can capture non-linear relationships.

(b) It models the relationship by transforming the independent variable into higher-degree terms (like X^2,X^3,..) and fits a polynomial curve that best represents the data pattern.

(c) The degree of a polynomial in Polynomial Regression is the highest power of the independent variable in the model.

(d) In Polynomial Regression, the coefficients of the polynomial equation are estimated using Least Squares Estimation (LSE).

(e) MSE is the average of the squared differences between the actual values and the predicted values. It is used to measure the error in a regression model.

(f) A lower MSE indicates that the model's predictions are closer to the actual data, meaning better accuracy.Model with the lowest MSE fits the data best.

(g) A polynomial with a higher degree can match the training data more closely, which may improve its performance on that data. But this also makes the model more complicated and increases the chance of overfitting.
lower-degree polynomial might be too simple and miss important trends, leading to underfitt

ing while the optimal fit balances the fit.

(h) We can overcome overfitting by  Choosing a lower-degree polynomial and using cross-validation to find the best polynomial degree.

(II)

a)

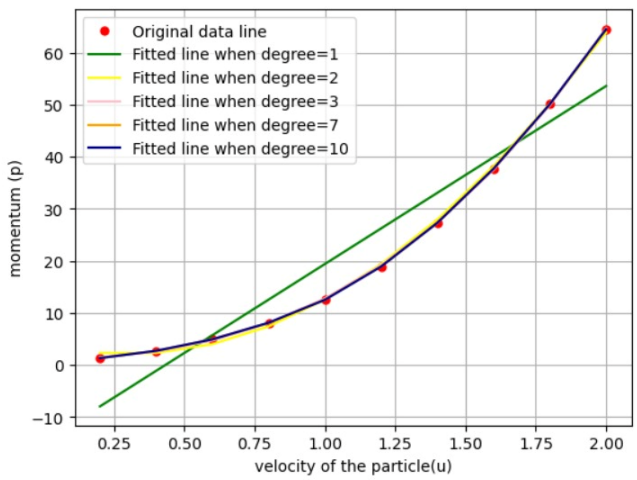
## a) best degree

[5]:
```
import numpy as np
import matplotlib.pyplot as plt
p= np.array([1.3,2.7,4.9,8.1,12.5,18.9,27.3,37.7,50.1,64.5])
u=np.linspace(0.2,2.0,10)
arr=[]
for i in range (0,9):
    a=i+1
    coe =np.polyfit(u,p,a)
    fit = np.polyval(coe,u)
    mse=np.mean((p-fit)**2)
    print("MSE ",a,"=",mse)
    arr.append(mse)
print("Best degree of polynomial for the above data : ",(np.argmin(arr)+1))
```

```
MSE  1 = 39.0688
MSE  2 = 0.3748606060606062
MSE  3 = 0.034853146853147186
MSE  4 = 0.00338461538461537
MSE  5 = 0.002871794871794811
MSE  6 = 0.0006839160839161356
MSE  7 = 0.0006530645824762928
MSE  8 = 0.00024886877827707805
MSE  9 = 1.7797343426887994e-21
Best degree of polynomial for the above data :  9
```

b)

```
[6]: import numpy as np
     import matplotlib.pyplot as plt
     p= np.array([1.3,2.7,4.9,8.1,12.5,18.9,27.3,37.7,50.1,64.5])
     u=np.linspace(0.2,2.0,10)
```




```
plt.show()
```

According to the above graph we can clearly see blue color curve is 2 closer to the original data points. So 10 degree polynomial is more accurate compare to other curves.

## c) Equation of best fit

```
p= np.array([1.3,2.7,4.9,8.1,12.5,18.9,27.3,37.7,50.1,64.5])
u=np.linspace(0.2,2.0,10)

# best fit degree is 10
coe =np.polyfit(u,p,10)
pol=np.poly1d(coe)
print(coe)
print (pol)
```

```
[   20.62034037  -167.61856607   486.15290518  -366.39691745
 -1202.00233081  3609.2331362  -4470.20697374  3041.99134293
 -1152.29159383   229.85636309   -16.83770587]
       10         9         8         7        6        5        4
20.62 x  - 167.6 x + 486.2 x - 366.4 x - 1202 x + 3609 x - 4470 x
          3        2
 + 3042 x - 1152 x + 229.9 x - 16.84
```

d)

## d) Value of p

[4]:
```
from sympy import symbols
x = symbols('x')
expression = 22.02*x**10 - 161*x**9+405.4*x**8-185.3*x**7
1105*x**6+2605*x**5-2705*x**4+1527*x**3+72.55*x-2.653
R = expression.subs(x, 0.9*3*10**8)
print("The result is:", R)
```

```
The result is: 4.53372260595205e+85
```

[ ]:

# 4. Lagrange Polynomial interpolation

(I)

a) The main idea is to create a single polynomial that passes exactly through all given data points**.**

b)

## Q4 Lagrange Polynomial interpolation

[ ]:
```
# (1) b)
```

[5]:
```
import numpy as np
from scipy.interpolate import lagrange
x =np.array([0,1,2,3,4,5,6])
y = x**4

p=lagrange(x,y)
print(p)
```

```
            6             5     4             3             2
2.22e-16 x + 7.105e-15 x + 1 x - 1.705e-13 x + 4.547e-13 x
```

c) L(x), and as an interpolation function, it should have the property L(x i)= yi for every point in the data set allowing the final polynomial to pass exactly through each point. For computing Lagrange polynomials, it is useful to write them as a linear combination of Lagrange basis polynomials.

d) Because each Lagrange basis polynomial L(xi) is **1** at xi and **0** at all other xj (for j≠i ),This makes the polynomial pass exactly through all data points**.**

e) The Lagrange polynomial provides a smooth function that can estimate values between known data points.It is the polynomial of lowest degree that assumes at each value the corresponding value, so that the functions coincide at each point.

f) Lagrange Polynomial Interpolation becomes complex when the number of data points increases.And it is sensitive to noise and can produce large oscillations, especially with high-degree polynomials . It is not ideal for extrapolation or very large data sets.

(II)

a)

```
[6]: # (2) a)
```

```
[9]: import numpy as np
     from scipy.interpolate import lagrange
     t =np.array([0,1,2,3,4,5])
     h =np.array([0,5,20,45,80,125])

     p=lagrange(t,h)
     print(p)
```

```
            5              3      2
4.441e-16 x - 1.421e-14 x + 5 x - 3.553e-15 x
```

b)

```
[13]: import numpy as np
      from scipy.interpolate import lagrange
      t =np.array([0,1,2,3,4,5])
      h =np.array([0,5,20,45,80,125])
      coe=np.polyfit(t,h,5)
      print(coe)
```

```
[-8.51307463e-16  1.16722050e-14 -5.75371210e-14  5.00000000e+00
 -9.58238304e-14  4.47065522e-15]
```

```
[ ]:
```

c)

```
[1]: # (2) c)
```

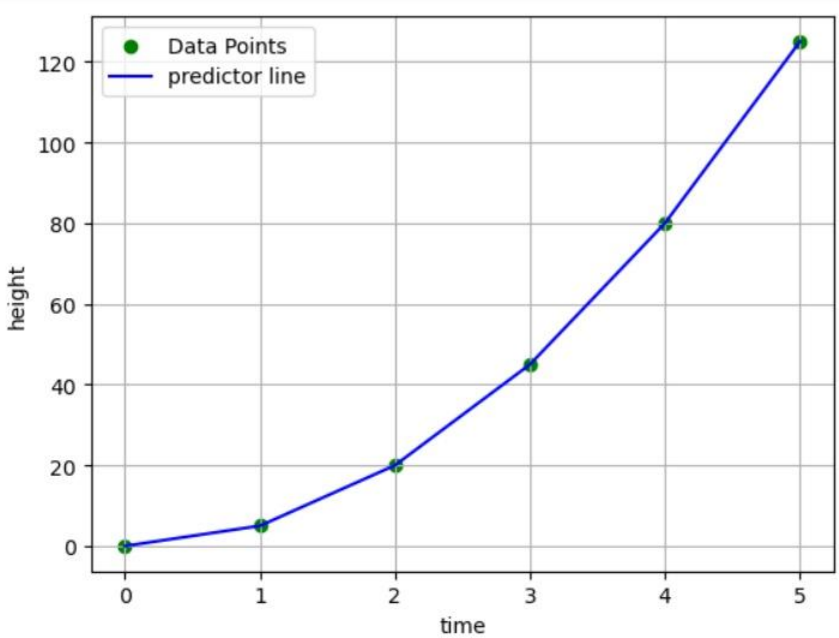
```
[11]: import numpy as np
 from scipy.interpolate import lagrange
 t = np.array([0, 1, 2,3,4,5])
 h = np.array([0, 5, 20, 45, 80, 125])
polynomial = lagrange(t, h)
 y1 = polynomial(t)
 Er= np.mean((h-y1) ** 2)
 print("Mean Square Error is :", Er)
```

```
Mean Square Error is : 2.1188803914236377e-25
```

d)

```
[4]: # (2) d)
```

```
[9]: import numpy as np
 from scipy.interpolate import lagrange
import matplotlib.pyplot as plt
 t = np.array([0, 1, 2,3,4,5])
 h = np.array([0, 5, 20, 45, 80, 125])
polynomial = lagrange(t, h)
 y1 = polynomial(t)

plt.scatter(t,h,label='Data Points',color='g')
plt.plot(t,y1,color='blue',label='predictor line')
plt.xlabel("time")
plt.ylabel("height")
plt.legend()
plt.grid(True)
plt.show()
```



e)

```
[10]: # (2) e)
```

```
[12]: from sympy import symbols
x = symbols('x')
ex=4.441e-16*x**5-1.421e-14*x**3 + 5*x**2-3.553e-15*x
R=ex.subs(x,2.5)
print("The velocity of the ball is ",R)
```

```
The velocity of the ball is  31.2499999999998
```

# 5. Linear Interpolation

(I)

a) Linear interpolation assumes that the data changes at a constant rate between two known points.It assumes a straight-line relationship between the values in that interval. By connecting two adjacent data points with a straight line, we can make a reasonable estimation of the value that falls between them.

b) It connects two adjacent known points with a straight line and estimates the value at a given point by finding the value on that line. The formula used is based on the slope between two known data points, typically represented as (x1, y1) and (x0, y0) points:
$(y-y0)/(x-x0) = (y1-y0)/(x1-x0)$
This helps calculate the unknown y value for any x within the interval.

c) No, linear interpolation cannot be used for values outside the known range. It can only be used within the interval between the two known points because it doesn't account for changes beyond that range.

d) When the distancs between target points decreases **(**closer the target point is to the known data points) , interpolation becomes more accurate . If the point is far from the known values, the accuracy decreases.

e) Linear interpolation uses straight lines, so it cannot capture curves or complex patterns. If the actual relationship between points is nonlinear, linear interpolation can give inaccurate results.

f) We can check if the data points lie approximately on a straight line. If the change between values appears linear and consistent, linear interpolation is suitable. Otherwise, for curves or uneven patterns more advanced methods like polynomial are better.

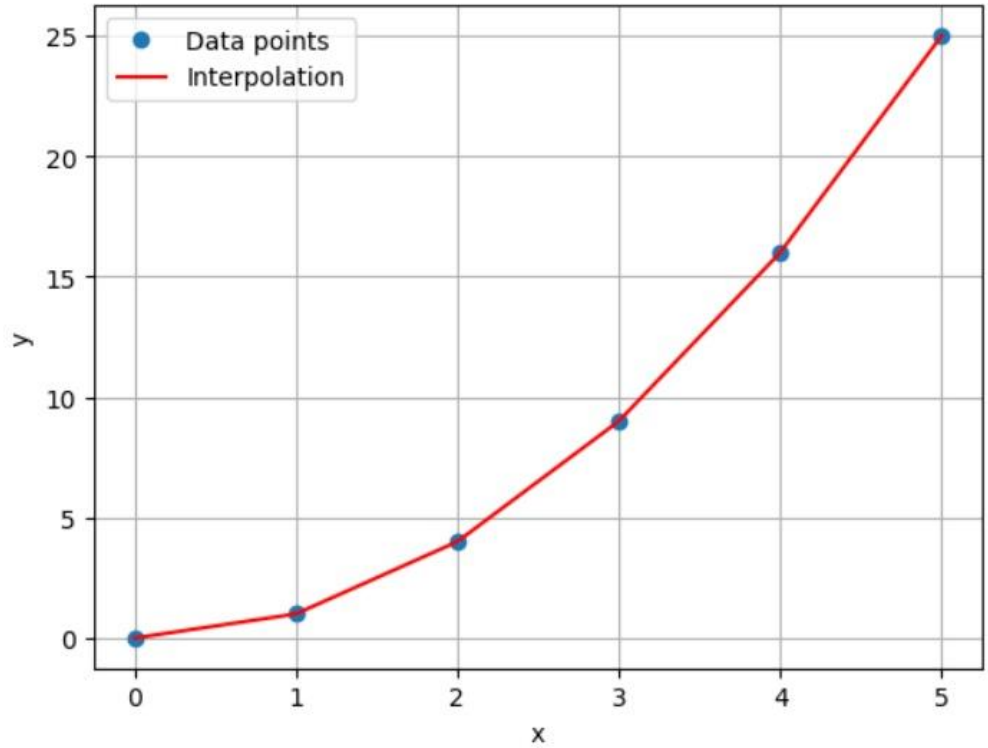
(II)

## Q5

[4]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d

x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([0, 1, 4, 9, 16, 25])

interpolation = interp1d(x, y, kind='linear', fill_value='extrapolate')
print(interpolation)

x1 = np.linspace(0, 5, 100)
y2 = interpolation(x1)

plt.plot(x, y, 'o', label='Data points')
plt.plot(x1, y2, label='Interpolation',color='red')
plt.legend()
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True)
plt.show()
```

```
<scipy.interpolate._interpolate.interp1d object at 0x00000202C1AB2C10>
```

```
<scipy.interpolate._interpolate.interp1d object at 0x00000202C1AB2C10>
```




Name :H.M.T.H.Lenadora
Reg. Num :2022s19441
Index Num: s16820